UNIT I · DATA WAREHOUSING

Inside the Cube

Making Sense of Data — From Every Angle at Once

A short visual tour of OLAP: how a business question turns into a data cube, how analysts slice and dice it, and how that cube actually gets built.

3+
TYPICAL DIMENSIONS

2^n
POSSIBLE CUBOIDS

5
CORE OLAP MOVES

Why Flat Tables Fall Short

A relational table answers one question well. A manager rarely asks only one.

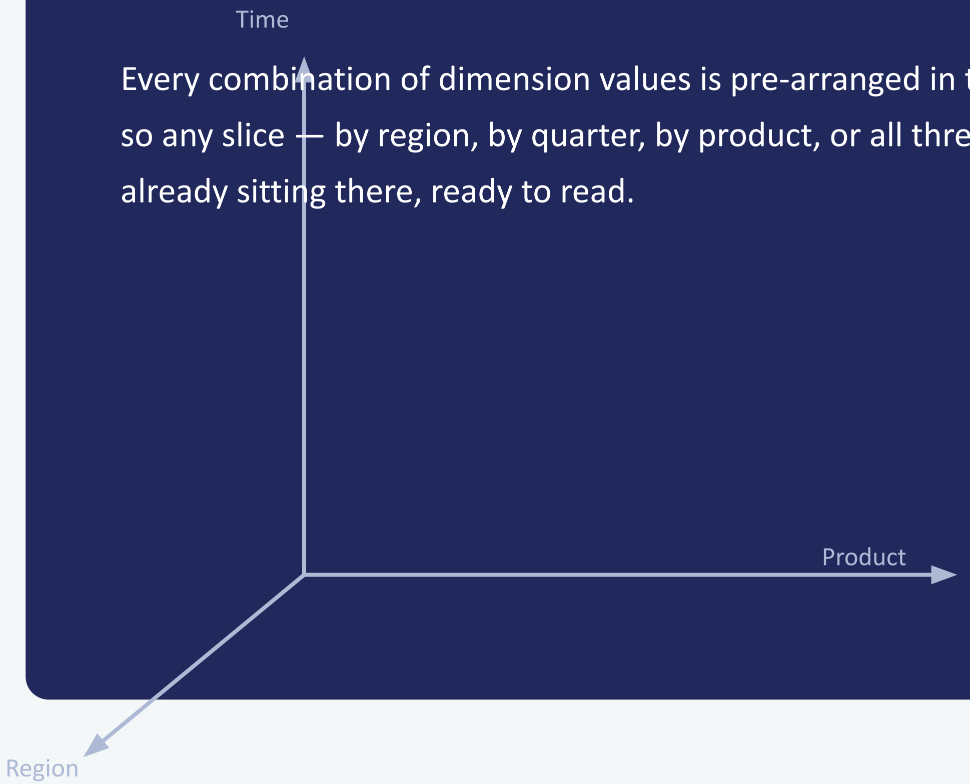
ONE TABLE, ONE VIEW

```
SELECT SUM(sales)
FROM transactions
WHERE region = 'South'
  AND quarter = 'Q3'
```

Change the question — a new product line, a different quarter, both at once — and it's a new query, written and run from scratch, every time.

ONE CUBE, EVERY ANGLE

Every combination of dimension values is pre-arranged in the cube, so any slice — by region, by quarter, by product, or all three — is already sitting there, ready to read.



Anatomy of a Data Cube

Every cube is built from two ingredients: dimensions to look through, and a measure to look at.

1

Dimensions

The perspectives you browse by — time, product, region, customer. Each has its own hierarchy (e.g. Day → Month → Quarter → Year).

2

Measures

The numbers you actually care about — sales, revenue, units sold. Stored in the fact table at the centre of the model.

3

Cells

Every dimension combination gives one cube cell holding an aggregated value — e.g. Sales, Q3, South, Footwear.

Five Moves of OLAP

Analysts don't write new queries — they nudge the same cube in different directions.

S Slice

Fix one dimension — e.g. Q3 only — to view a single flat layer of the cube.

D Dice

Filter two or more dimensions at once to carve out a smaller sub-cube.

↓ Drill-down

Move from a summary level to finer detail — Year into Quarter into Month.

↑ Roll-up

Climb the hierarchy the other way — Month rolled up into Year totals.

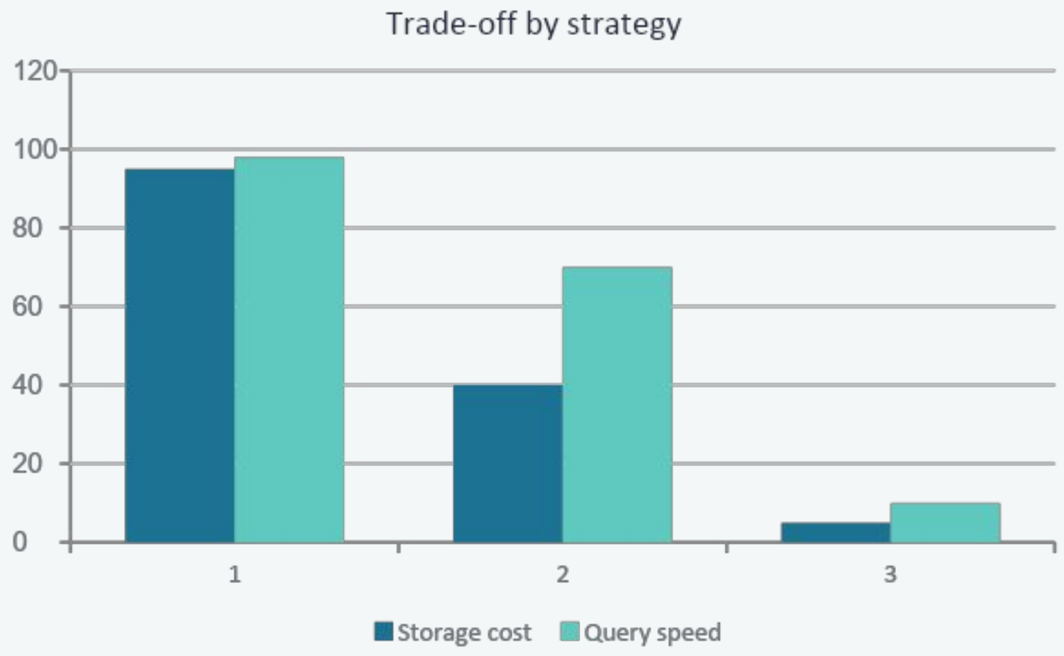
⟳ Pivot

Rotate the cube's axes to view the same data from a new orientation.

Same cube.
Five lenses.
Zero re-queries.

Building the Cube: Computation Methods

A cube with n dimensions has 2^n possible cuboids — pre-computing all of them is rarely worth it.



Full materialization

Compute and store every cuboid. Fastest queries, but storage explodes as dimensions grow.

Iceberg cube

Only pre-compute cells above a minimum support threshold — skip combinations nobody queries.

Partial / on-demand

Compute the base cuboid only; higher-level cuboids are aggregated live, on request.

The Big Picture

A data cube doesn't answer one question — it holds every answer at once, along every dimension, so a manager can slice, dice, drill and pivot without ever going back to the database.

1. **Model it** — Fact table + dimension tables define the cube's shape.
2. **Move through it** — Slice, dice, roll-up, drill-down and pivot reshape the view instantly.
3. **Compute it wisely** — Full, iceberg, or on-demand — choose based on what's actually queried.

Thank you